

浙江大学设计报告

课程名称： 人工智能 任课教师： 杨倩倩 上课时间： 周四 345 节

项目选题： 基于 Multi-agent 的自动化数据分析与报告生成流

小组成员：

姓名	学号	学院	专业	班级
----	----	----	----	----

一、项目概述

本次项目选题为基于 Multi-agent 的自动化数据分析与报告生成流。本项目实现了一个面向多表数据（CSV / Excel / ZIP 输入）的自动化数据分析系统。用户可上传多份表格文件或一个包含多张表的 ZIP 压缩包，系统基于多智能体协作完成以下任务：

- ①多表数据画像与质量审计
- ②候选关联键自动识别
- ③联合分析与统计（描述统计、分组、相关性、时间趋势等）
- ④图表生成
- ⑤视觉模型图表质量审查
- ⑥结构化报告输出

本系统不依赖启发式 fallback，所有智能体决策与代码生成均基于真实大模型运行。

二、项目核心目标

本项目预期解决的核心问题为，当用户提供多张原始数据表和一句自然语言分析需求时，系统自动完成从数据理解到联合分析报告生成的完整流程。围绕这个目标，当前版本已经支持的功能有：

- ①多文件或 ZIP 输入
- ②多表画像与关联键识别
- ③数据清洗与联合分析
- ④图表生成与视觉审查
- ⑤结构化结果输出
- ⑥反思修复闭环

三、多智能体架构

本系统采用多智能体协作架构，将复杂的多表数据分析任务拆解为规划、路由、编码、执行、校验、反思、视觉审查与报告生成等可独立运作又相互协作的子任务。整体工作流如图 1 所示，Agent 的具体分工如表 1 所示。

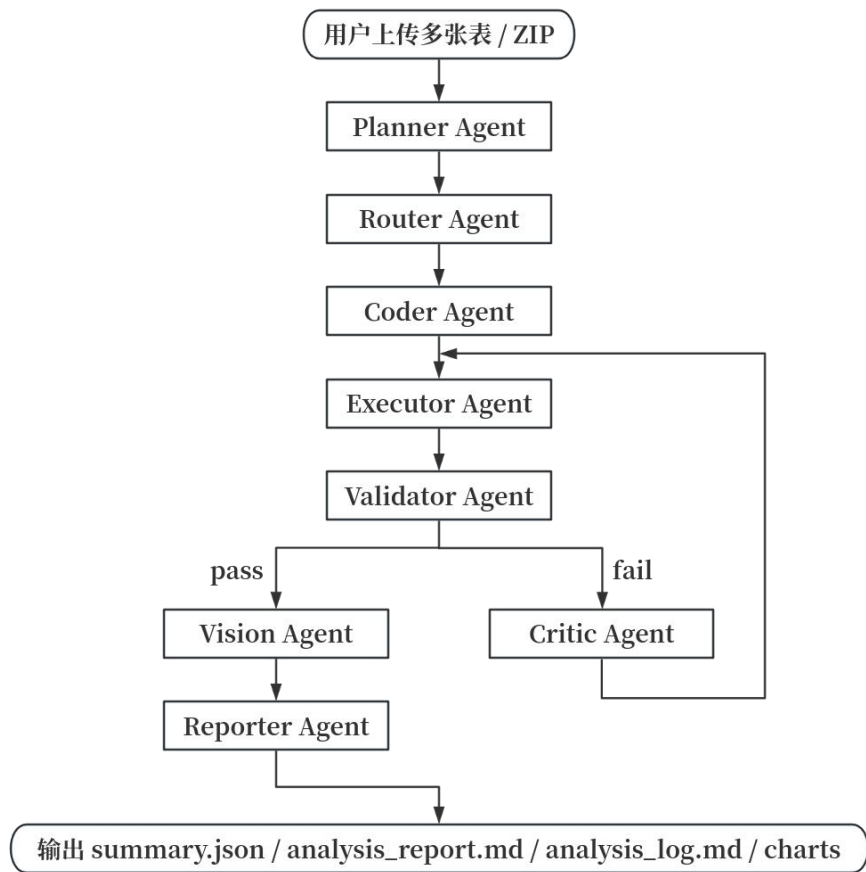


图 1 整体 workflow

智能体	职责
Planner Agent	根据多表画像和分析目标生成任务图、候选关联策略和分析重点
Router Agent	将任务图转为可执行批次，明确结果校验重点
Coder Agent	生成支持多表加载、关联和联合分析的完整 Python 脚本
Executor Agent	执行代码，产出图表、日志和结构化中间结果
Validator Agent	校验运行结果是否满足输出要求（不仅脚本跑通）
Critic Agent	当执行失败或结果不合格时，结合报错与反馈修复代码
Vision Agent	使用视觉模型检查图表可读性和表达质量
Reporter Agent	将结构化分析结果整理为报告上下文，调用 LLM 生成正式报告

表 1 Agent 分工

四、当前能力边界

目前的系统已支持：

- ①多个 CSV/ Excel 文件或 ZIP 压缩包输入
- ②多表候选关联键识别
- ③单事实表+维表类联合分析
- ④通用数据质量审计
- ⑤保守型清洗
- ⑥多表联合统计分析
- ⑦图表生成

⑧反思修复

⑨图表视觉审查

目前的系统暂不支持：

①高复杂度多跳连接自动优化

②非结构化/半结构化复杂数据源

③专门的预测建模流水线

④任务级细粒度执行器

⑤自动数据血缘追踪

因此，当前版本的系统已经具备“多表协同分析”的基础能力，但还不是一个完整的数据中台或 AutoML 平台。

五、项目结构

目前的系统的项目结构如图 2 所示。

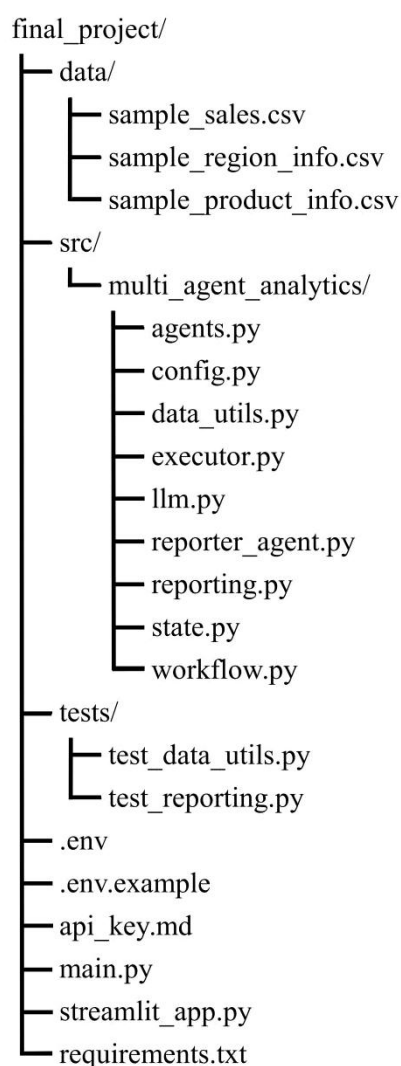


图 2 项目结构

六、系统配置方式

1、准备工作

使用该系统需要在电脑上安装 Python 软件，并将整个系统文件夹复制到电脑上。

2、安装依赖

在命令行窗口中找到文件地址，并输入以下命令，即可完成依赖安装。

```
pip install -r requirements.txt
```

3、创建配置文件

本系统需要调用大模型 API 来完成任务，因此必须获得合法的 API 密钥。

系统将按以下顺序读取模型配置：命令行参数→项目目录中的.env 文件→项目目录中的 api_key.md 文件

对于.env 文件，使用时需要在 final_project 文件夹中找到.env.example 文件，将其重命名为.env，并修改成以下内容：

```
CHAT_API_KEY=your_deepseek_key
```

```
CHAT_BASE_URL=https://api.deepseek.com
```

```
CHAT_MODEL=deepseek-v4-flash
```

```
VISION_API_KEY=your_qwen_key
```

```
VISION_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
```

```
VISION_MODEL=qwen3.5-flash
```

```
MAX_REFLECTIONS=2
```

对于 api_key.md 文件，使用时需要在 final_project 文件夹中新建一个名为 api_key.md 的文件，并写入以下内容：

```
## deepseek
```

```
api_key: your_key
```

```
model: deepseek-v4-flash
```

```
base_url: https://api.deepseek.com
```

```
## Qwen_VL
```

```
api_key: your_key
```

```
model: qwen3.5-flash
```

```
base_url: https://dashscope.aliyuncs.com/compatible-mode/v1
```

七、系统运行方式

1、命令行运行

使用命令行窗口运行该系统时，基本的命令格式如下：

```
python main.py
```

```
--data 文件 1 文件 2 ...
```

```
--question "您的分析需求"
```

下面给出几种具体的命令模版。

①上传多个分散文件

```
python main.py \
```

```
--data data/sample_sales.csv data/sample_region_info.csv data/sample_product_info.csv \
```

```
--question "请对多表数据进行联合分析并输出结构化报告。"
```

②上传 ZIP 压缩包

```
python main.py \
```

```
--data data_bundle.zip \
```

```
--question "请识别多表关联并完成联合分析。"
```

③指定输出目录

```
python main.py \  
--data data/sample_sales.csv data/sample_region_info.csv data/sample_product_info.csv \  
--question "分析销售表现" \  
--output outputs/my_multi_table_run
```

2、Streamlit 界面运行

Streamlit 界面是一个可视化界面，无需手动输入复杂命令，只需在界面中选择文件并输入分析需求即可。

在命令行窗口中输入以下命令，便可打开 Streamlit 界面。

```
python -m streamlit run streamlit_app.py
```

目前，Streamlit 界面支持：

- ①上传多个 CSV / Excel 文件
- ②上传一个 ZIP 文件
- ③使用项目内置多表示例数据
- ④自动预览每张表
- ⑤展示候选关联关系
- ⑥运行多表协同分析
- ⑦查看关联决策、图表、报告和结构化结果

八、系统输出

每次运行后，系统会在输出目录生成：

```
analysis_report.md;  
analysis_log.md;  
summary.json;  
charts/*.png;  
runtime/analysis_attempt_*.py;  
runtime/stdout_attempt_*.log;  
runtime/stderr_attempt_*.log。
```

其中 summary.json 是结构化机器输出；analysis_report.md 是 Reporter Agent 调用 LLM 直接生成的正式书面分析报告，默认使用中文，只有在分析需求中明确要求英文时才会输出英文报告；analysis_log.md 保存多智能体执行细节、校验结果、连接决策与运行错误，供调试和复核。

当前版本中，正式报告不再由固定模板代码拼接生成，而是由 Reporter Agent 基于结构化分析结果调用大模型直接撰写。

报告生成入口位于 src/multi_agent_analytics/reporter_agent.py；正式报告的核心 prompt 位于 reporter_node() 中的 system_prompt，如果需要手动调整报告语言、章节结构、写作风格或多表整合叙事方式，优先修改这个 prompt；src/multi_agent_analytics/reporting.py 当前只负责将正式报告写入 analysis_report.md，并将技术日志写入 analysis_log.md。

九、反思与校验机制

当前版本的反思机制同时考虑运行状态与结果质量。Validator Agent 会检查：

- ①是否生成 summary.json
- ②是否包含 task_results

- ③是否包含 data_quality
- ④是否包含 findings
- ⑤对多表任务是否包含 table_overview
- ⑥对多表任务是否包含 join_analysis
- ⑦若规划要求出图，是否真的生成图表

若这些条件不满足，系统会将当前代码、运行错误日志、校验失败原因、多表画像、候选连接信息、任务规划一起反馈给 Critic Agent 进行修复。

十、项目亮点

本项目的价值不只是“调用了大模型写代码”，而是把大模型纳入一个可执行、可验证、可反思的多表分析流程中。它的核心亮点在于：

- ①多智能体分工
- ②多表画像与候选关联识别
- ③结果校验与反思修复
- ④多模态图表审查
- ⑤面向复现的结构化输出

十一、项目示例

为了演示本项目，展示项目的功能与优点，项目内置了一个最小多表样例，包含销售事实表 sample_sales.csv、区域维表 sample_region_info.csv、产品维表 sample_product_info.csv。在输入样例后，系统会自动识别表格并将可用于关联的数据进行连接，即将文件 sample_sales.csv 和 sample_region_info.csv 中的 region 列数据，以及文件 sample_sales.csv 和 sample_product_info.csv 中的 product 列数据进行连接，并据此进行多表联合分析。

上述样例使用真实 API 完成了多表端到端测试，成功验证了本系统可正常识别多表输入、可自动发现候选连接、可成功执行联合分析、可成功生成图表与报告、可进行视觉模型审查，系统各功能正常，可以正常使用。

十二、系统后续扩展方向

目前本系统仅具备“多表协同分析”的基础能力，如果想要继续往更完整的通用数据整理分析系统推进，下一步最值得做的是：

- ①更严格的主外键推断与连接评分
- ②支持多跳关联图而不只是简单维表连接
- ③中间表产物注册与血缘追踪
- ④任务级细粒度代码执行
- ⑤自动业务规则校验器
- ⑥视觉审查失败后的自动重绘闭环

十三、项目总结

本项目设计并实现了一个基于 Multi-Agent 的自动化数据分析与报告生成系统。用户只需上传多个 CSV/Excel 表格或一个包含多张表的 ZIP 压缩包，并输入自然语言分析需求，系统即可自动完成从数据理解到联合分析报告生成的完整流程。

系统采用多智能体分工架构，包含规划（Planner）、路由（Router）、编码（Coder）、执行（Executor）、校验（Validator）、反思（Critic）、视觉审查（Vision）和报告生成（Reporter）等智能体。各智能体协同工作，依次完成多表数据画像与质量审计、候选关联键自动识别、

联合分析与统计、图表生成、视觉模型图表质量审查，最终输出结构化报告。

当前系统已支持多文件或 ZIP 输入、多表画像与关联键识别、数据清洗与联合分析、图表生成与视觉审查、结构化结果输出以及反思修复闭环。系统提供了命令行和 Streamlit 可视化界面两种运行方式，并支持通过配置文件灵活接入大模型 API（如 DeepSeek、千问）。每次运行会生成分析报告、执行日志、结构化 JSON 结果以及图表文件，便于调试与复现。

项目核心价值在于将大模型能力纳入一个可执行、可验证、可反思的多表分析流程中，而非简单调用大模型生成代码。通过校验与反思机制，系统能够自动修复执行失败或结果不合格的情况，提高了自动化分析的鲁棒性和实用性。

十四、小组感想

本次项目是我们小组在人工智能课程中的一次综合性实践。从选题到最终实现，我们经历了多次讨论、分工协作和反复调试，收获颇丰。

首先，在技术层面，我们深入理解了多智能体系统的设计思想，学会了如何将复杂的多表数据分析任务拆解为可独立运作的子任务，并通过智能体之间的协作完成整体目标。同时，我们也掌握了如何调用大模型 API 进行代码生成、图表审查和报告撰写，并设计了校验与反思机制来弥补大模型输出不稳定的问题。这些经验对我们今后从事更复杂的智能化系统开发具有重要启发。

其次，在团队协作方面，四位组员分别承担了不同的开发工作，过程中也遇到了诸如多表关联逻辑设计、校验条件设定、视觉模型调用失败等困难，但通过集体攻关和查阅资料，最终都得以解决。这次项目让我们深刻体会到，良好的沟通与合理的任务分解是团队成功的关键。

最后，我们认识到当前系统仍有很大的扩展空间，例如加强主外键推断、支持多跳关联图等。未来如果有机会，我们希望继续完善这个项目，使其成为一个更通用的数据智能分析工具。感谢老师的指导，以及课程提供的学习平台，让我们能够在实践中提升人工智能应用开发能力。